

1 Exercise 1 Interpolation

a)/b)/c)

We started the simulation with *stage_ros* and planned a path to visualize the A* algorithm and the calculated smoothed path as shown in figure 1



Figure 1: Visualization of A* and smoothed path in Rviz

2 Exercise 2 Orthogonal projection

a)

For each frame, the robot determines the nearest orthogonal projection onto a pose of the smoothed path. This is done by finding the closest pose on the path based on the Euclidean distance.

b)

We define the sign of the projection based on the relative position of the robot to the path. This is determined using the perpendicular dot product between the vector from the robot to the nearest path point and the tangent vector of the path. A positive value indicates the robot is on one side of the path, while a negative value means it is on the other side.

c)

We compute the control angle ϕ_c in the robot's coordinate system. Added to the angle θ_t of the path tangent this results in the desired angle of the robot. This angle is used to adjust the robot's heading towards the path, ensuring smooth path-following.

3 Exercise 3 PID

a)

A **PID controller** (Proportional-Integral-Derivative controller) is a control system used to maintain a desired output by continuously adjusting an input variable based on feedback. It consists of three components:

- **Proportional (P):** Reacts to the present error by adjusting the control variable proportionally to the difference between the desired value (setpoint) and the actual value.
- **Integral (I):** Accounts for past errors by integrating the error over time and applying corrections to eliminate steady-state errors.
- **Derivative (D):** Predicts future errors by analyzing the rate of change of the error and applying corrections to prevent overshooting.

The **time-continuous formula** for a PID controller is:

$$u(t) = K_p e(t) + K_i \int e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

where:

- $u(t)$ is the control output,
- $e(t)$ is the error (difference between setpoint and actual value),
- K_p , K_i , and K_d are the proportional, integral, and derivative gains.

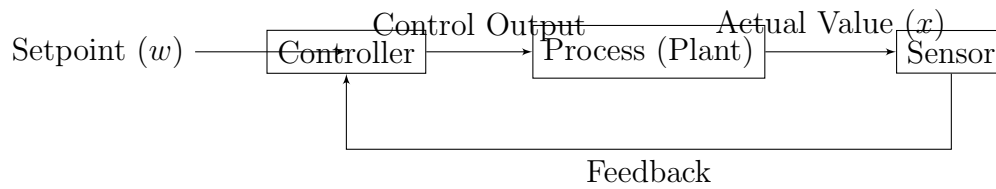
The **time-discrete formula** (used in digital implementations) is:

$$u_k = K_p e_k + K_i \sum e_k \Delta t + K_d \frac{e_k - e_{k-1}}{\Delta t}$$

where k represents the discrete time step, and Δt is the sampling interval.

PID controllers are widely used in robotics, automation, and industrial applications to ensure stable and precise control.

b)



c)

We've implemented the PID Controller as a class Structure.

d)

The robot is following the given smoothed path.

e)

We followed a path in the simulation:

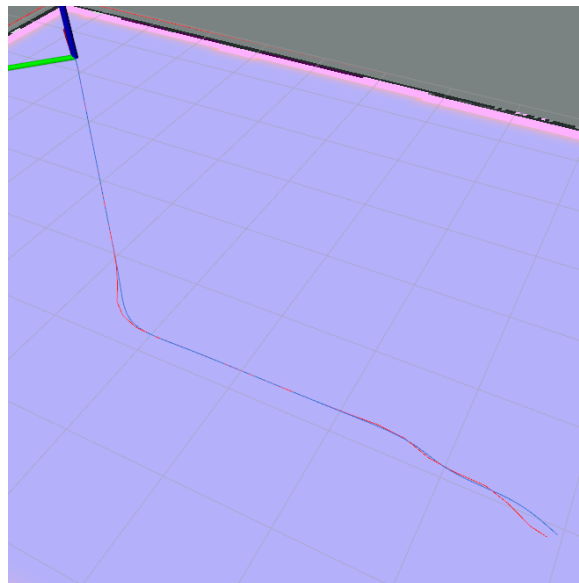


Figure 2: PID controlled path following in simulation

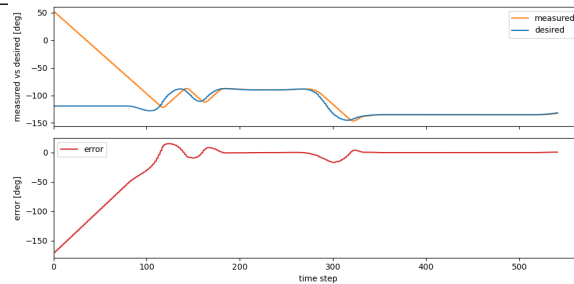


Figure 3: measured and desired angle and error of path following in the simulation

f)

4 Exercise 4: Navigation

a)

We tested the program on the real robot, and it successfully followed the planned path.

b)

We integrated the mapping node with the cost map, ensuring that the generated map is correctly published and used for path planning.

c)

The robot successfully follows the planned path.

d)

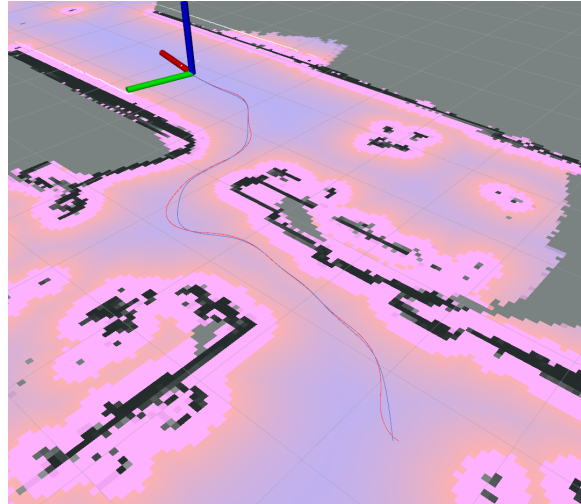


Figure 4: Pathplanning with the Costmap in the simulation

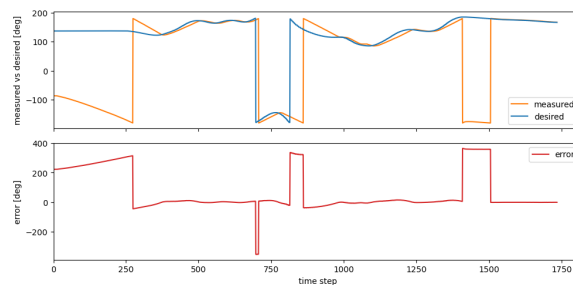


Figure 5: Pathplanning with the Costmap in the simulation

e)

We integrated the obstacle avoidance approach into the navigation system. The robot switches to the obstacle avoidance node if it is in the warning zone of the Costmap (we check if the robot's current position or a position along the future path (50 poses ahead of the path → approx. 1 meter) grid cell cost is above 50), if yes, switch to obstacle avoidance, if no stay in path following node and continue following the planned path.

When no obstacles are nearby, the standard path following behavior is resumed.

f)

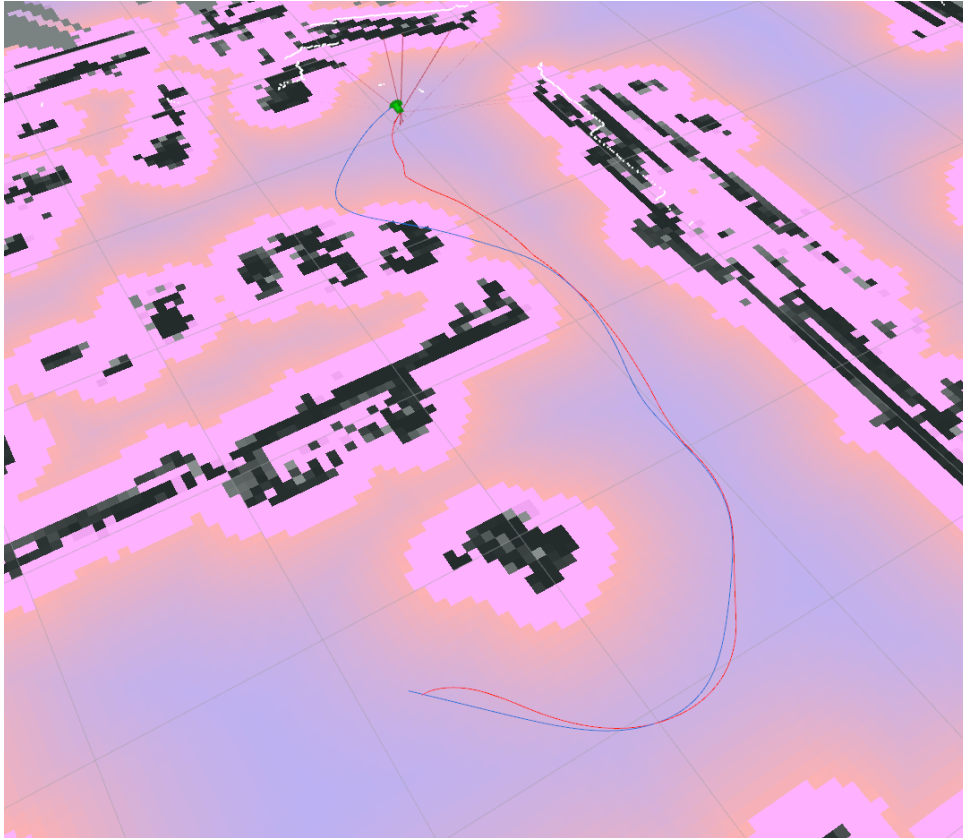


Figure 6: Actual and the desired path of the real robot in Rviz